

Data Analysis with Python

Lesson 2: Python Basics for Data Analysis

Python Basics for Data Analysis - Study Notes

Key Concepts

1. **Variables & Data Types** – Storing and manipulating numbers, strings, booleans, and complex data structures.
2. **Control Flow** – Using `if`, `elif`, `else`, `for`, and `while` loops to drive logic and iterate over data.
3. **Functions** – Encapsulating reusable code blocks, accepting arguments, and returning values.
4. **List Comprehensions** – Concise, readable way to create and transform lists, often used for quick data transformations.
5. **Basic I/O & Data Import** – Reading from files or standard input to feed data into Python for analysis.

Summary

Python's simplicity makes it a popular choice for data analysis. At the core are variables that hold data of various types—integers, floats, strings, booleans, and more complex structures like lists, tuples, dictionaries, and sets. Understanding how to manipulate these types is essential for cleaning and preparing data.

Control flow structures (`if` statements, loops) allow you to process datasets conditionally and iteratively. Functions let you package repetitive tasks into reusable units, improving readability and maintainability. List comprehensions provide a powerful, expressive syntax for creating new lists from existing iterables, often replacing verbose `for` loops.

Together, these fundamentals enable you to read raw data, transform it, and perform preliminary analyses—all before moving on to more advanced libraries such as NumPy, pandas, or matplotlib.

Important Points

- **Variable Naming**: Use descriptive names (`age`, `sales_total`) and follow snake_case convention.
- **Mutable vs Immutable**: Lists, dictionaries, and sets are mutable; tuples and strings are immutable.
- **Type Conversion**: Use `int()`, `float()`, `str()`, etc., to convert between types when needed.
- **Loop Syntax**:

```
```python
for item in iterable:
 # process item
```
```

- **List Comprehension Syntax**:

```
```python
```

```
new_list = [expression for item in iterable if condition]
```

```
...
```

- **Function Definition**:

```
```python
```

```
def function_name(param1, param2='default'):
```

```
    # body
```

```
    return result
```

```
...
```

- **Return Values**: Functions can return multiple values as tuples.
- **Docstrings**: Add `"""Docstring"""` immediately after the function definition to describe its purpose.
- **Error Handling**: Use `try/except` blocks when reading files or converting types to avoid crashes.
- **PEP 8**: Follow style guidelines for readability (indentation, line length, etc.).

```
---
```

Examples

1. Reading a CSV and Summing a Column

```
```python
```

```
import csv
```

```
def sum_column(file_path, column_name):
```

```
 total = 0
```

```
 with open(file_path, newline='') as csvfile:
```

```
 reader = csv.DictReader(csvfile)
```

```
 for row in reader:
```

```
 total += float(row[column_name])
```

```
 return total
```

## Usage

```
sales_total = sum_column('sales.csv', 'revenue')
```

```
print(f"Total revenue: ${sales_total:,.2f}")
```

```
...
```

**\*Explanation\*:**

- `csv.DictReader` reads each row as a dictionary keyed by column names.
- The loop converts the string value to `float` and accumulates it.
- The function returns the total, which is then formatted for display.

### 2. List Comprehension for Data Cleaning

```
```python
```

```
raw_data = [' 10', '20', ' 30', 'forty', '50 ']
```

```
clean_numbers = [int(item.strip()) for item in raw_data if item.strip().isdigit()]
```

```
print(clean_numbers) # Output: [10, 20, 30, 50]
```

```
...
```

***Explanation*:**

- ``item.strip()`` removes surrounding whitespace.
- ``isdigit()`` filters out non-numeric entries.
- The comprehension converts each valid string to an integer in one line.

Quick Review

1. ****What is the difference between a list and a tuple in Python?****
2. ****Write a function that takes a list of numbers and returns the average.****
3. ****Convert the following list comprehension into an equivalent `for` loop:****

```
```python  
squares = [x**2 for x in range(1, 6)]
```
```
4. ****Explain why you might prefer a list comprehension over a traditional loop.****
5. ****What built in function would you use to check if a variable is of type `dict`?****

Further Reading

- ****NumPy Basics**** – Arrays, vectorized operations, broadcasting.
- ****pandas Fundamentals**** – DataFrames, indexing, filtering, and aggregation.
- ****Python Standard Library**** – ``itertools``, ``functools``, ``collections`` for advanced data manipulation.
- ****Data Visualization**** – ``matplotlib``, ``seaborn``, or ``plotly`` for visualizing results.
- ****Error Handling & Logging**** – Robust code with ``try/except``, ``logging`` module.

These topics build on the foundations covered here and will prepare you for real world data analysis projects.

